

Building an AI Software Engineering Team

A Step-by-Step Implementation Guide with LangGraph, MCP, FastAPI & PostgreSQL

What you'll build: a multi-agent system where a Product Manager, Software Architect, Backend Developer, Frontend Developer, QA Engineer, and Code Reviewer — each an LLM-driven agent — collaborate through a stateful graph to turn a natural-language specification into a working, tested, reviewed codebase.

Stack covered: LangGraph · Claude / OpenAI / Gemini · MCP (Model Context Protocol) · FastAPI · PostgreSQL · Docker

Table of Contents

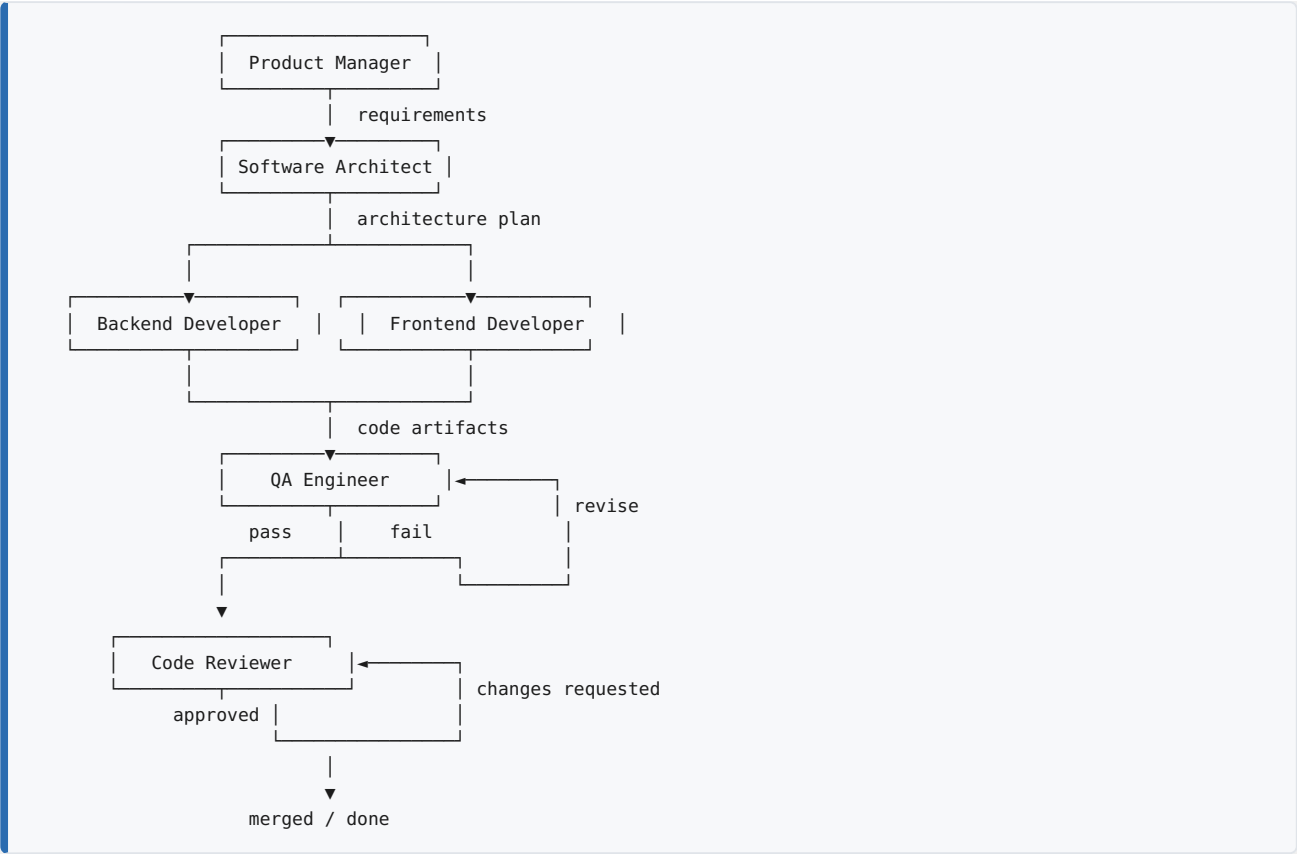
1. Architecture Overview
 2. Prerequisites & Environment Setup
 3. Project Structure
 4. Part A — PostgreSQL & Persistence Layer
 5. Part B — Shared Graph State
 6. Part C — Tool Layer via MCP (Git, GitHub, Docker Sandbox, Python Exec, Browser, Docs RAG)
 7. Part D — The Six Agents (prompts + code for each)
 8. Part E — Assembling the LangGraph Graph (routing & revision loops)
 9. Part F — The Docker Code-Execution Sandbox
 10. Part G — FastAPI Service Layer
 11. Part H — Running It End-to-End
 12. Part I — Dockerized Deployment (docker-compose)
 13. Part J — Observability & Debugging
 14. Common Pitfalls & Best Practices
 15. Appendix — Full File Tree & Config Files
-

1. Architecture Overview

1.1 Why a graph, not a pipeline

A naive implementation chains agents in a straight line: PM → Architect → Backend Dev → Frontend Dev → QA → Reviewer → done. Real software development is not linear — QA fails and work returns to the developer; a reviewer rejects a diff and sends it back; the architect's plan sometimes needs revision once developers hit a technical wall. This system is modeled as a **directed graph with cycles**, which is exactly what LangGraph is designed for (as opposed to a simple LCEL chain, which only supports linear or DAG-shaped flows without native support for conditional loops and durable state).

1.2 High-level flow



Each node in this diagram is a Python function (or subgraph) registered in a `langgraph.graph.StateGraph`. Edges are either fixed (`add_edge`) or conditional (`add_conditional_edges`), and the conditional edges are what implement the QA-fail and review-rejected loops.

1.3 Component responsibilities

Component	Role	Key inputs	Key outputs
Product Manager	Turn a vague spec into concrete, testable requirements & user stories	Raw natural-language spec	Structured requirements doc (JSON)
		Requirements	

Component	Role	Key inputs	Key outputs
Software Architect	Decide tech stack, module boundaries, API contracts, DB schema		Architecture plan (JSON): file tree, endpoints, schema
Backend Developer	Implement server-side code, APIs, DB models	Architecture plan	File diffs / new files (backend)
Frontend Developer	Implement UI/client code against the API contract	Architecture plan	File diffs / new files (frontend)
QA Engineer	Write & run tests against the implementation	Backend + frontend code	Pass/fail + logs + bug reports
Code Reviewer	Static review for quality, security, consistency	Code diffs + test results	Approve, or list of requested changes

1.4 Tool layer

Agents don't just talk — they act. Tools are exposed uniformly through **MCP (Model Context Protocol)** servers so any agent, and you during debugging, can call them the same way:

Tool	Used by	Purpose
Git	Backend/Frontend Dev, Reviewer	branch, commit, diff
GitHub	Reviewer, orchestrator	open PRs, post review comments, track issues
Docker	QA Engineer	run generated code and tests in an isolated sandbox
Python execution	Backend Dev, QA	run scripts, execute tests, lint
Browser (Playwright)	Frontend Dev, QA	render pages, screenshot, click-test the UI
Documentation retrieval (RAG)	all dev agents	look up library/API docs so generated code uses real, current APIs

1.5 Persistence

LangGraph's **Postgres checkpointer** stores the entire graph state after every node execution. This buys you three things for free:

1. **Crash recovery** — resume a run exactly where it left off.
2. **Human-in-the-loop** — pause before a risky step (e.g. merging to main) and resume after approval.
3. **Auditability** — replay any past run to see exactly what every agent produced and why.

We'll also keep a separate lightweight `projects` table for your own application-level metadata (status, repo URL, owner), independent of LangGraph's internal checkpoint tables.

2. Prerequisites & Environment Setup

2.1 Required accounts / keys

- An LLM API key: Anthropic (`ANTHROPIC_API_KEY`), OpenAI (`OPENAI_API_KEY`), and/or Google (`GOOGLE_API_KEY`) — you can mix providers per agent.
- A GitHub Personal Access Token with `repo` scope (for the GitHub MCP tool).
- Docker Desktop (or Docker Engine on Linux) installed and running.
- PostgreSQL 15+ (local via Docker, or a hosted instance).

2.2 System requirements

- Python 3.11 or 3.12
- Node.js 18+ (only needed if you use the official TypeScript-based MCP servers for GitHub/Git; Python MCP servers are also available)
- 8GB+ RAM recommended — you will run an LLM client, a Postgres container, and one or more sandboxed Docker containers for code execution simultaneously

2.3 Create the project and virtual environment

```
mkdir ai-swe-team && cd ai-swe-team
python -m venv venv

# Windows PowerShell
.\venv\Scripts\Activate.ps1
# macOS / Linux
source venv/bin/activate
```

2.4 Install core dependencies

```
pip install langgraph langchain langchain-anthropic langchain-openai \
    langchain-mcp-adapters langgraph-checkpoint-postgres \
    fastapi "uvicorn[standard]" psycopy[binary] psycopy-pool \
    docker gitpython pydantic python-dotenv \
    chromadb playwright
```

Then install the Playwright browser binaries (needed for the Browser tool):

```
playwright install chromium
```

2.5 requirements.txt

Freeze this early so the environment is reproducible:

```
pip freeze > requirements.txt
```

2.6 .env file

Create a `.env` in the project root:

```
# LLM providers
ANTHROPIC_API_KEY=sk-ant-...
OPENAI_API_KEY=sk-...
GOOGLE_API_KEY=...

# GitHub
GITHUB_TOKEN=ghp-...
GITHUB_DEFAULT_ORG=your-org-or-username

# Postgres
POSTGRES_HOST=localhost
POSTGRES_PORT=5432
POSTGRES_DB=ai_swe_team
POSTGRES_USER=postgres
POSTGRES_PASSWORD=postgres
DATABASE_URL=postgresql://postgres:postgres@localhost:5432/ai_swe_team

# Sandbox
SANDBOX_IMAGE=ai-swe-sandbox:latest
SANDBOX_NETWORK_DISABLED=true
```

Load it at the top of every entrypoint:

```
from dotenv import load_dotenv
load_dotenv()
```

2.7 Start Postgres quickly with Docker

```
docker run --name ai-swe-postgres -d \
-e POSTGRES_PASSWORD=postgres \
-e POSTGRES_DB=ai_swe_team \
-p 5432:5432 \
postgres:16
```

3. Project Structure

Lay the project out so agents, tools, graph wiring, and the API are cleanly separated. This structure is referenced throughout the rest of the guide.

```
ai-swe-team/
├── .env
├── requirements.txt
├── docker-compose.yml
├── Dockerfile.sandbox          # image used for sandboxed code execution
├── app/
│   ├── main.py                # FastAPI app entrypoint
│   ├── api/
│   │   ├── routes_projects.py # /projects endpoints
│   │   └── schemas.py         # Pydantic request/response models
│   ├── graph/
│   │   ├── state.py           # ProjectState TypedDict
│   │   ├── build_graph.py     # StateGraph wiring, routing functions
│   │   └── checkpointer.py    # Postgres checkpointer setup
│   ├── agents/
│   │   ├── product_manager.py
│   │   ├── architect.py
│   │   ├── backend_dev.py
│   │   ├── frontend_dev.py
│   │   ├── qa_engineer.py
│   │   └── code_reviewer.py
│   ├── tools/
│   │   ├── mcp_config.py      # MCP server definitions
│   │   ├── sandbox.py         # Docker sandbox runner
│   │   ├── git_tool.py
│   │   ├── docs_retrieval.py  # RAG over library docs
│   │   └── browser_tool.py
│   ├── db/
│   │   ├── models.py          # SQLAlchemy models for projects table
│   │   ├── session.py
│   │   └── migrations/        # Alembic migrations
│   └── config.py               # settings loaded from .env
├── tests/
│   └── test_graph.py
└── scripts/
    ├── init_db.py
    └── run_sample_project.py
```

4. Part A — PostgreSQL & Persistence Layer

Two separate concerns live in Postgres:

1. **LangGraph checkpoints** — internal state snapshots, managed by `langgraph-checkpoint-postgres`. You don't design this schema; the library creates and manages its own tables (`checkpoints` , `checkpoint_writes` , etc.) the first time you call `.setup()` .
2. **Your application metadata** — a `projects` table you control, for status tracking, listing runs, linking to repos.

4.1 Application schema

`app/db/models.py` :

```
from sqlalchemy import Column, String, DateTime, Text, Integer
from sqlalchemy.orm import declarative_base
from datetime import datetime
import uuid

Base = declarative_base()

class Project(Base):
    __tablename__ = "projects"

    id = Column(String, primary_key=True, default=lambda: str(uuid.uuid4()))
    thread_id = Column(String, unique=True, nullable=False) # links to LangGraph checkpoint thread
    name = Column(String, nullable=False)
    spec = Column(Text, nullable=False)
    status = Column(String, default="pending") # pending|running|awaiting_review|done|failed
    repo_url = Column(String, nullable=True)
    revision_count = Column(Integer, default=0)
    created_at = Column(DateTime, default=datetime.utcnow)
    updated_at = Column(DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)
```

`app/db/session.py` :

```
import os
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker

engine = create_engine(os.environ["DATABASE_URL"])
SessionLocal = sessionmaker(bind=engine, autoflush=False, autocommit=False)

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

`scripts/init_db.py` — run once to create tables:

```

from dotenv import load_dotenv
load_dotenv()
from app.db.session import engine
from app.db.models import Base

Base.metadata.create_all(engine)
print("Application tables created.")

```

For anything beyond a prototype, replace the last two lines with **Alembic** migrations so schema changes are versioned:

```

pip install alembic
alembic init app/db/migrations
# edit alembic.ini's sqlalchemy.url, then:
alembic revision --autogenerate -m "create projects table"
alembic upgrade head

```

4.2 LangGraph checkpointer

app/graph/checkpointer.py :

```

import os
from psychopg_pool import ConnectionPool
from langgraph.checkpoint.postgres import PostgresSaver

def get_checkpointer() -> PostgresSaver:
    pool = ConnectionPool(
        conninfo=os.environ["DATABASE_URL"],
        max_size=20,
        kwargs={"autocommit": True, "prepare_threshold": 0},
    )
    checkpointer = PostgresSaver(pool)
    checkpointer.setup() # creates LangGraph's internal tables if they don't exist
    return checkpointer

```

Call `get_checkpointer()` once at startup and pass the instance into `graph.compile(checkpointer=checkpointer)` (shown in Part E).

5. Part B — Shared Graph State

Every node reads from and writes to one shared state object. Getting this schema right early prevents painful refactors later, since every agent prompt and every routing function depends on it.

app/graph/state.py :

```
from typing import TypedDict, List, Dict, Optional, Annotated
import operator

class FileArtifact(TypedDict):
    path: str
    content: str
    language: str

class TestResult(TypedDict):
    passed: bool
    total: int
    failed: int
    failures: List[str]
    logs: str

class ReviewComment(TypedDict):
    file: str
    line: Optional[int]
    severity: str          # "blocking" | "suggestion" | "nit"
    comment: str

class ProjectState(TypedDict):
    # --- input ---
    spec: str

    # --- PM output ---
    requirements: str
    user_stories: List[str]

    # --- Architect output ---
    architecture: Dict      # {"stack": [...], "file_tree": [...], "api_contracts": [...], "db_schema": {...}}

    # --- Developer outputs ---
    backend_files: Dict[str, FileArtifact]
    frontend_files: Dict[str, FileArtifact]

    # --- QA outputs ---
    test_results: Optional[TestResult]

    # --- Review outputs ---
    review_comments: List[ReviewComment]
    review_approved: bool

    # --- control flow ---
    revision_count: int
    max_revisions: int
    status: str              # "planning"|"developing"|"testing"|"reviewing"|"done"|"failed"

    # --- trace / observability ---
    messages: Annotated[List, operator.add]
```

Design notes:

- `messages` uses `Annotated[list, operator.add]` — LangGraph's reducer pattern. Instead of each node overwriting the field, LangGraph *appends* whatever a node returns for that key. This gives you a full transcript of every agent's reasoning without any node needing to know about prior entries.
 - All other fields use the default "last write wins" behavior, which is what you want for structured artifacts like `architecture` or `test_results`.
 - `revision_count` / `max_revisions` are what prevent infinite QA-fail loops (see Part E, section 8.4).
 - Keep artifacts as **structured dicts**, not raw prose. Every agent should return JSON-parseable output; free-text explanations go in `messages`, not in the fields that downstream agents / your routing logic depend on.
-

6. Part C — Tool Layer via MCP

Rather than hand-rolling tool integrations per agent, expose every tool as an MCP server. This gives you one consistent calling convention across Git, GitHub, Docker, Python execution, browser automation, and documentation retrieval — and it means you can swap or upgrade a tool's implementation without touching agent code.

6.1 MCP client configuration

app/tools/mcp_config.py :

```
import os
from langchain_mcp_adapters.client import MultiServerMCPClient

def build_mcp_client() -> MultiServerMCPClient:
    return MultiServerMCPClient({
        "git": {
            "command": "mcp-server-git",
            "args": ["--repository", os.environ.get("REPO_PATH", ".")],
            "transport": "stdio",
        },
        "github": {
            "url": "https://api.githubcopilot.com/mcp/",
            "transport": "sse",
            "headers": {"Authorization": f"Bearer {os.environ['GITHUB_TOKEN']}"},
        },
        "docs": {
            "command": "python",
            "args": ["-m", "app.tools.docs_mcp_server"],
            "transport": "stdio",
        },
    })

async def get_tools_for(agent_names: list[str]):
    """Fetch only the MCP tools relevant to a given agent, by server name."""
    client = build_mcp_client()
    all_tools = await client.get_tools()
    # langchain-mcp-adapters names tools like "<server>_<tool>"
    return [t for t in all_tools if any(t.name.startswith(f"{n}__") for n in agent_names)]
```

Install the reference MCP servers:

```
# Git MCP server (Python, from the official MCP servers repo)
pip install mcp-server-git

# GitHub's MCP server is hosted remotely (api.githubcopilot.com/mcp) -
# no local install needed, just an Authorization header with a GitHub PAT.
```

6.2 Docker as a tool (sandboxed execution)

The Docker "tool" is not exposed as a generic shell — that would let an agent run arbitrary commands on your host. Instead, wrap it as a narrow function tool that only runs code **inside a locked-down container** using the Python `docker` SDK. Full implementation in Part F; here's the LangChain tool wrapper agents call:

```
# app/tools/sandbox.py (interface used by agents; full runner in Part F)
from langchain_core.tools import tool
from app.tools.sandbox_runner import run_in_sandbox

@tool
def run_python_tests(code_files: dict[str, str], test_command: str = "pytest -q") -> dict:
    """Run the given files and test command inside an isolated, network-disabled
    Docker container. Returns {passed, total, failed, failures, logs}."""
    return run_in_sandbox(code_files, test_command)
```

6.3 Browser tool (Playwright)

app/tools/browser_tool.py :

```
from langchain_core.tools import tool
from playwright.sync_api import sync_playwright

@tool
def render_and_screenshot(url: str, wait_selector: str | None = None) -> dict:
    """Load a URL in a headless browser, optionally wait for a CSS selector,
    and return a base64 screenshot plus any console errors. Used by QA and
    the Frontend Developer to visually verify rendered pages."""
    console_errors = []
    with sync_playwright() as p:
        browser = p.chromium.launch()
        page = browser.new_page()
        page.on("console", lambda msg: console_errors.append(msg.text) if msg.type == "error" else None)
        page.goto(url, timeout=15000)
        if wait_selector:
            page.wait_for_selector(wait_selector, timeout=10000)
        screenshot_bytes = page.screenshot(full_page=True)
        browser.close()
    import base64
    return {
        "screenshot_b64": base64.b64encode(screenshot_bytes).decode(),
        "console_errors": console_errors,
    }
```

6.4 Documentation retrieval (RAG)

Agents writing code hallucinate library APIs less when they can look up real docs first. A small ChromaDB-backed retrieval tool:

```
# app/tools/docs_retrieval.py
import chromadb
from chromadb.utils import embedding_functions
from langchain_core.tools import tool
import os

_client = chromadb.PersistentClient(path="./chroma_docs")
_embed_fn = embedding_functions.OpenAIEmbeddingFunction(
    api_key=os.environ["OPENAI_API_KEY"], model_name="text-embedding-3-small"
)
_collection = _client.get_or_create_collection("library_docs", embedding_function=_embed_fn)

@tool
def search_docs(query: str, n_results: int = 5) -> list[str]:
    """Search indexed library/framework documentation for relevant snippets."""
    results = _collection.query(query_texts=[query], n_results=n_results)
    return results["documents"][0] if results["documents"] else []
```

Populate `_collection` ahead of time with docs for whatever frameworks your generated projects will use (FastAPI, React, SQLAlchemy, etc.) — chunk the official docs and `.add()` them once, offline, before running the agent team.

6.5 Binding tools to a specific agent

LangChain's `bind_tools` attaches a tool set to one model instance so that model can invoke tools and the graph node parses the tool calls:

```
from langchain_anthropic import ChatAnthropic

llm = ChatAnthropic(model="claude-sonnet-4-6")
backend_llm = llm.bind_tools([run_python_tests, search_docs])
```

Each agent module (Part D) only binds the tools it actually needs — the PM and Architect typically need none; the developers need sandbox execution and docs retrieval; QA needs sandbox execution and the browser tool; the Reviewer needs Git/GitHub.

7. Part D — The Six Agents

Each agent lives in its own module: a system prompt, a structured-output schema, and a node function with the signature `(state: ProjectState) -> dict` (LangGraph merges the returned dict into the shared state).

Shared pattern used by every agent — force structured JSON output using `with_structured_output`, so downstream nodes and routing functions never have to parse free text:

```
from pydantic import BaseModel, Field
from typing import List
```

7.1 Product Manager

`app/agents/product_manager.py` :

```
from pydantic import BaseModel, Field
from typing import List
from langchain_anthropic import ChatAnthropic
from app.graph.state import ProjectState

class Requirements(BaseModel):
    requirements: str = Field(description="Clear, structured functional & non-functional requirements")
    user_stories: List[str] = Field(description="User stories in 'As a ... I want ... so that ...' form")

PM_SYSTEM_PROMPT = """You are a senior Product Manager on a software team.
Given a raw, possibly vague natural-language project specification, produce:
1. Clear functional requirements (what the system must do)
2. Clear non-functional requirements (performance, security, scale, if implied)
3. A set of concrete user stories

Be concrete. Resolve ambiguity by making explicit, reasonable assumptions and
stating them. Do not design the technical solution – that is the Architect's job.
"""

llm = ChatAnthropic(model="claude-sonnet-4-6").with_structured_output(Requirements)

def product_manager_node(state: ProjectState) -> dict:
    result: Requirements = llm.invoke([
        ("system", PM_SYSTEM_PROMPT),
        ("human", f"Project specification:\n\n{state['spec']}"),
    ])
    return {
        "requirements": result.requirements,
        "user_stories": result.user_stories,
        "status": "requirements_done",
        "messages": [f"[PM] Produced {len(result.user_stories)} user stories."],
    }
```

7.2 Software Architect

`app/agents/architect.py` :

```

from pydantic import BaseModel, Field
from typing import List, Dict
from langchain_anthropic import ChatAnthropic
from app.graph.state import ProjectState

class ArchitecturePlan(BaseModel):
    stack: List[str] = Field(description="Concrete technology choices, e.g. 'FastAPI', 'PostgreSQL', 'React+Vite'")
    file_tree: List[str] = Field(description="Planned file/module paths, e.g. 'backend/app/main.py'")
    api_contracts: List[Dict] = Field(description="List of {method, path, request_schema, response_schema}")
    db_schema: Dict = Field(description="Table/collection definitions, keyed by name")
    rationale: str = Field(description="Short justification of key design decisions")

ARCHITECT_SYSTEM_PROMPT = """You are a senior Software Architect.
Given product requirements, design a concrete, buildable technical plan:
- Choose a specific tech stack appropriate to the requirements' scale and constraints
- Define the file/module structure the developers will implement against
- Specify REST API contracts precisely enough that backend and frontend
  developers can work in parallel without conflicting
- Define the database schema
Favor simple, well-supported technology choices over novel ones unless the
requirements specifically demand otherwise.
"""

llm = ChatAnthropic(model="claude-sonnet-4-6").with_structured_output(ArchitecturePlan)

def architect_node(state: ProjectState) -> dict:
    result: ArchitecturePlan = llm.invoke([
        ("system", ARCHITECT_SYSTEM_PROMPT),
        ("human", f"Requirements:\n{state['requirements']}\n\nUser stories:\n{state['user_stories']}"),
    ])
    return {
        "architecture": result.model_dump(),
        "status": "architecture_done",
        "messages": [f"[Architect] Chose stack: {' '.join(result.stack)}"],
    }

```

7.3 Backend Developer

app/agents/backend_dev.py :

```

from pydantic import BaseModel, Field
from typing import List
from langchain_anthropic import ChatAnthropic
from app.graph.state import ProjectState, FileArtifact
from app.tools.docs_retrieval import search_docs

class CodeFile(BaseModel):
    path: str
    content: str
    language: str

class BackendOutput(BaseModel):
    files: List[CodeFile]
    notes: str = Field(description="Anything the QA engineer or reviewer should know")

BACKEND_SYSTEM_PROMPT = """You are a senior Backend Developer.
Implement the backend exactly according to the given architecture plan:
API contracts, database schema, and file structure. Write complete,
runnable code, not snippets or pseudocode. Include:
- All route handlers matching the specified API contracts
- Database models/migrations matching the schema
- A requirements.txt or equivalent dependency list
- At least basic input validation and error handling

If review or QA feedback is included below, address every point before
returning new files.
"""

llm = ChatAnthropic(model="claude-sonnet-4-6").with_structured_output(BackendOutput)

def backend_dev_node(state: ProjectState) -> dict:
    # Pull relevant docs for the chosen stack to reduce API hallucination
    doc_snippets = []
    for tech in state["architecture"].get("stack", []):
        doc_snippets.extend(search_docs.invoke({"query": tech, "n_results": 3}))

    feedback = ""
    if state.get("test_results") and not state["test_results"]["passed"]:
        feedback += f"\nQA failures to fix:\n{state['test_results']['failures']}"
    if state.get("review_comments"):
        blocking = [c for c in state["review_comments"] if c["severity"] == "blocking"]
        if blocking:
            feedback += f"\nReviewer blocking comments to address:\n{blocking}"

    result: BackendOutput = llm.invoke([
        ("system", BACKEND_SYSTEM_PROMPT),
        ("human",
         f"Architecture plan:\n{state['architecture']}\n\n"
         f"Relevant docs:\n{doc_snippets}\n\n"
         f"Feedback to address (empty on first pass):\n{feedback}"),
    ])

    files = {f.path: FileArtifact(path=f.path, content=f.content, language=f.language) for f in result.files}
    return {
        "backend_files": files,
        "status": "backend_done",
        "messages": [f"[Backend Dev] Wrote {len(files)} files. {result.notes}"],
    }

```

7.4 Frontend Developer

`app/agents/frontend_dev.py` follows the identical pattern, swapping the prompt focus to UI implementation against the architect's API contracts:

```

from pydantic import BaseModel, Field
from typing import List
from langchain_anthropic import ChatAnthropic
from app.graph.state import ProjectState, FileArtifact

class CodeFile(BaseModel):
    path: str
    content: str
    language: str

class FrontendOutput(BaseModel):
    files: List[CodeFile]
    notes: str

FRONTEND_SYSTEM_PROMPT = """You are a senior Frontend Developer.
Implement the client application against the architecture's API contracts.
Write complete, runnable code (components, routing, API client, styles).
Match the requirements' user stories to concrete UI flows. If review or QA
feedback is included, address every point before returning new files.
"""

llm = ChatAnthropic(model="claude-sonnet-4-6").with_structured_output(FrontendOutput)

def frontend_dev_node(state: ProjectState) -> dict:
    feedback = ""
    if state.get("review_comments"):
        blocking = [c for c in state["review_comments"] if c["severity"] == "blocking"]
        if blocking:
            feedback = f"Reviewer blocking comments to address:\n{blocking}"

    result: FrontendOutput = llm.invoke([
        ("system", FRONTEND_SYSTEM_PROMPT),
        ("human",
         f"Architecture / API contracts:\n{state['architecture']}\n\n"
         f"User stories:\n{state['user_stories']}\n\n"
         f"Feedback (empty on first pass):\n{feedback}"),
    ])

    files = {f.path: FileArtifact(path=f.path, content=f.content, language=f.language) for f in result.files}
    return {
        "frontend_files": files,
        "status": "frontend_done",
        "messages": [f"[Frontend Dev] Wrote {len(files)} files. {result.notes}"],
    }

```

7.5 QA Engineer

This is the first agent that actually **executes** code, via the sandboxed Docker tool from Part F.

app/agents/qa_engineer.py :

```

from app.graph.state import ProjectState, TestResult
from app.tools.sandbox import run_python_tests

QA_SYSTEM_PROMPT = """You are a QA Engineer. You have been given backend and
(optional) frontend source files. Write a pytest test suite covering the
architecture's API contracts and the PM's user stories, then execute it."""

def qa_engineer_node(state: ProjectState) -> dict:
    # 1. Ask an LLM to generate tests targeting the API contracts (omitted
    # here for brevity – same with_structured_output pattern as above,
    # returning {"test_files": {path: content}}).
    from app.agents._qa_test_writer import generate_tests
    test_files = generate_tests(state)

    # 2. Merge implementation + test files and execute inside the sandbox
    all_files = {**{p: f["content"] for p, f in state["backend_files"].items()},
                 **test_files}
    raw_result = run_python_tests.invoke({"code_files": all_files, "test_command": "pytest -q"})

    result = TestResult(
        passed=raw_result["passed"],
        total=raw_result["total"],
        failed=raw_result["failed"],
        failures=raw_result["failures"],
        logs=raw_result["logs"],
    )
    return {
        "test_results": result,
        "status": "qa_passed" if result["passed"] else "qa_failed",
        "messages": [f"[QA] {result['total']-result['failed']}/{result['total']} tests passed."],
    }

```

7.6 Code Reviewer

app/agents/code_reviewer.py :

```

from pydantic import BaseModel, Field
from typing import List
from langchain_anthropic import ChatAnthropic
from app.graph.state import ProjectState, ReviewComment

class ReviewOutput(BaseModel):
    approved: bool
    comments: List[dict] = Field(description="List of {file, line, severity, comment}")

REVIEWER_SYSTEM_PROMPT = """You are a senior Code Reviewer. Review the given
backend and frontend files for: correctness against the architecture plan,
security issues (injection, secrets in code, missing auth checks), code
quality, and consistency with the passed test results. Classify each issue's
severity as 'blocking', 'suggestion', or 'nit'. Approve only if there are no
blocking issues."""

llm = ChatAnthropic(model="claude-sonnet-4-6").with_structured_output(ReviewOutput)

def code_reviewer_node(state: ProjectState) -> dict:
    all_files = {**state["backend_files"], **state["frontend_files"]}
    result: ReviewOutput = llm.invoke([
        ("system", REVIEWER_SYSTEM_PROMPT),
        ("human",
         f"Files:\n{all_files}\n\nTest results:\n{state['test_results']}"),
    ])
    comments = [ReviewComment(**c) for c in result.comments]
    return {
        "review_comments": comments,
        "review_approved": result.approved,
        "status": "approved" if result.approved else "changes_requested",
        "messages": [f"[Reviewer] {'Approved' if result.approved else f'{len(comments)} comments'}"],
    }

```

Model selection tip: the PM, Architect, and Reviewer benefit most from a stronger model (Claude Sonnet/Opus) since they make judgment calls that affect everything downstream. QA's test-writing step can often use a cheaper/faster model. Mixing providers per agent is straightforward — just swap the `llm = Chat...(...)` line in each module.

8. Part E — Assembling the LangGraph Graph

8.1 Build the graph

`app/graph/build_graph.py` :

```

from langgraph.graph import StateGraph, END
from app.graph.state import ProjectState
from app.graph.checkpointer import get_checkpointer

from app.agents.product_manager import product_manager_node
from app.agents.architect import architect_node
from app.agents.backend_dev import backend_dev_node
from app.agents.frontend_dev import frontend_dev_node
from app.agents.qa_engineer import qa_engineer_node
from app.agents.code_reviewer import code_reviewer_node

def route_after_qa(state: ProjectState) -> str:
    if state["test_results"]["passed"]:
        return "code_reviewer"
    if state["revision_count"] < state["max_revisions"]:
        return "revise"
    return "failed"

def route_after_review(state: ProjectState) -> str:
    if state["review_approved"]:
        return "done"
    if state["revision_count"] < state["max_revisions"]:
        return "revise"
    return "failed"

def increment_revision(state: ProjectState) -> dict:
    return {"revision_count": state["revision_count"] + 1}

def build_graph():
    graph = StateGraph(ProjectState)

    graph.add_node("pm", product_manager_node)
    graph.add_node("architect", architect_node)
    graph.add_node("backend_dev", backend_dev_node)
    graph.add_node("frontend_dev", frontend_dev_node)
    graph.add_node("qa", qa_engineer_node)
    graph.add_node("reviewer", code_reviewer_node)
    graph.add_node("bump_revision", increment_revision)

    graph.set_entry_point("pm")
    graph.add_edge("pm", "architect")

    # Backend and frontend run in parallel branches off the architect
    graph.add_edge("architect", "backend_dev")
    graph.add_edge("architect", "frontend_dev")

    # QA waits for both branches to complete (LangGraph fan-in: a node with
    # multiple incoming edges runs once all of them have completed)
    graph.add_edge("backend_dev", "qa")
    graph.add_edge("frontend_dev", "qa")

    graph.add_conditional_edges("qa", route_after_qa, {
        "code_reviewer": "reviewer",
        "revise": "bump_revision",
        "failed": END,
    })

    graph.add_conditional_edges("reviewer", route_after_review, {
        "done": END,
        "revise": "bump_revision",
        "failed": END,
    })

```

```

# After bumping the revision counter, send work back to the developers
graph.add_edge("bump_revision", "backend_dev")
graph.add_edge("bump_revision", "frontend_dev")

checkpointer = get_checkpointer()
return graph.compile(checkpointer=checkpointer)

```

8.2 Why the fan-out / fan-in works

LangGraph tracks, per **superstep**, which nodes are scheduled to run next. When `architect` finishes and both `backend_dev` and `frontend_dev` are added as its successors, LangGraph schedules both for the next superstep — that's your parallel branch. `qa` has incoming edges from *both* — LangGraph will only execute `qa` once every incoming branch for the current superstep has completed, giving you a synchronization/fan-in point for free.

8.3 Handling concurrent writes to the same state key

Because `backend_dev` and `frontend_dev` run concurrently and each writes to a different key (`backend_files` vs `frontend_files`), there's no conflict. If two parallel nodes ever need to write the **same** key, give that key a reducer (like `messages` does with `operator.add`) or LangGraph will raise an `InvalidUpdateError` at runtime — this is a deliberate safety check, not a bug.

8.4 Revision loop safety

`max_revisions` is set once at graph invocation (see 8.5) and never mutated inside the loop — only `revision_count` increases, via the dedicated `bump_revision` node. This guarantees termination: after `max_revisions` failed attempts, `route_after_qa` / `route_after_review` return `"failed"` and the graph ends rather than looping forever. Treat a `"failed"` terminal status as a signal for **human escalation**, not a silent dead end — surface it clearly in your FastAPI layer (Part G).

8.5 Invoking the graph

```

import uuid
from app.graph.build_graph import build_graph

app_graph = build_graph()

def start_project(spec: str) -> str:
    thread_id = str(uuid.uuid4())
    config = {"configurable": {"thread_id": thread_id}}
    initial_state = {
        "spec": spec,
        "revision_count": 0,
        "max_revisions": 3,
        "status": "planning",
        "review_comments": [],
        "messages": [],
    }
    app_graph.invoke(initial_state, config=config)
    return thread_id

```

Because a full run can take minutes, prefer `ainvoke` / `astream` from an async FastAPI route (Part G) over blocking `invoke` , and consider running the graph in a background task or worker process rather than inside the request/response cycle.

9. Part F — The Docker Code-Execution Sandbox

This is the highest-risk component: agent-generated code must **never** execute directly on your host. Every execution goes through a locked-down, ephemeral, network-disabled container.

9.1 Sandbox image

Dockerfile.sandbox :

```
FROM python:3.12-slim

RUN pip install --no-cache-dir pytest fastapi httpx sqlalchemy pydantic

# Non-root user — generated code should never run as root
RUN useradd -m -u 1000 sandboxuser
USER sandboxuser
WORKDIR /workspace

# No ENTRYPOINT/CMD — the runner injects the exact command per execution
```

Build it once:

```
docker build -t ai-swe-sandbox:latest -f Dockerfile.sandbox .
```

9.2 The sandbox runner

app/tools/sandbox_runner.py :

```

import docker
import tarfile
import io
import os

_client = docker.from_env()
SANDBOX_IMAGE = os.environ.get("SANDBOX_IMAGE", "ai-swe-sandbox:latest")

def _build_tar(files: dict[str, str]) -> io.BytesIO:
    buf = io.BytesIO()
    with tarfile.open(fileobj=buf, mode="w") as tar:
        for path, content in files.items():
            data = content.encode("utf-8")
            info = tarfile.TarInfo(name=path)
            info.size = len(data)
            tar.addfile(info, io.BytesIO(data))
    buf.seek(0)
    return buf

def run_in_sandbox(code_files: dict[str, str], test_command: str, timeout: int = 60) -> dict:
    """Execute `test_command` against `code_files` inside a fresh, isolated,
    network-disabled, resource-limited container. Always removes the
    container afterward, even on failure."""

    container = _client.containers.create(
        image=SANDBOX_IMAGE,
        command=["sleep", str(timeout + 5)], # keep alive; we exec into it
        working_dir="/workspace",
        network_disabled=True, # no outbound network access at all
        mem_limit="512m",
        nano_cpus=1_000_000_000, # 1 CPU
        pids_limit=128, # prevent fork bombs
        read_only=False, # workspace needs to be writable; root fs could be tightened further
        detach=True,
    )

    try:
        container.start()
        container.put_archive("/workspace", _build_tar(code_files))

        exit_code, output = container.exec_run(
            cmd=["bash", "-lc", test_command],
            workdir="/workspace",
            demux=False,
        )
        logs = output.decode("utf-8", errors="replace")

        return _parse_pytest_output(exit_code, logs)

    finally:
        container.kill()
        container.remove(force=True)

def _parse_pytest_output(exit_code: int, logs: str) -> dict:
    import re
    passed = exit_code == 0
    m = re.search(r"(\d+) passed", logs)
    total_passed = int(m.group(1)) if m else 0
    m = re.search(r"(\d+) failed", logs)
    total_failed = int(m.group(1)) if m else 0
    failures = re.findall(r"FAILED (\S+)", logs)
    return {
        "passed": passed,

```

```
"total": total_passed + total_failed,
"failed": total_failed,
"failures": failures,
"logs": logs[-4000:],    # cap log size returned to the LLM
}
```

9.3 Sandbox hardening checklist

- **network_disabled=True** — generated code cannot exfiltrate data or hit arbitrary URLs, even accidentally.
 - **mem_limit / nano_cpus / pids_limit** — bound resource usage so a runaway or malicious script can't affect the host or other containers.
 - **Non-root user** inside the image — even inside the container, code doesn't run as root.
 - **Always remove(force=True)** in a **finally** block — no orphaned containers accumulating on the host.
 - **Timeouts** — wrap **exec_run** with your own watchdog if a test could hang (e.g. run it in a thread with **concurrent.futures.ThreadPoolExecutor** and enforce a hard timeout, killing the container if exceeded).
 - **No secrets baked into the sandbox image or passed via environment** — QA doesn't need real API keys; use mock/stub credentials for anything the generated code might call out to (which it shouldn't be able to reach anyway, given **network_disabled**).
 - Consider **gVisor** (**runsc**) as the container runtime for an additional kernel-isolation layer if you'll run this against untrusted specs in production.
-

10. Part G — FastAPI Service Layer

FastAPI is the control surface: trigger runs, check status, stream progress, and eventually serve as the place where a human approves a "failed"/escalated run.

10.1 Pydantic schemas

app/api/schemas.py :

```
from pydantic import BaseModel
from typing import Optional

class CreateProjectRequest(BaseModel):
    name: str
    spec: str

class ProjectResponse(BaseModel):
    id: str
    thread_id: str
    name: str
    status: str
    revision_count: int
    repo_url: Optional[str] = None
```

10.2 Routes

app/api/routes_projects.py :

```

import uuid
from fastapi import APIRouter, BackgroundTasks, Depends, HTTPException
from sqlalchemy.orm import Session

from app.api.schemas import CreateProjectRequest, ProjectResponse
from app.db.session import get_db
from app.db.models import Project
from app.graph.build_graph import build_graph

router = APIRouter(prefix="/projects", tags=["projects"])
graph = build_graph()

@router.post("", response_model=ProjectResponse)
async def create_project(req: CreateProjectRequest, background_tasks: BackgroundTasks, db: Session = Depends(get_db)):
    thread_id = str(uuid.uuid4())
    project = Project(thread_id=thread_id, name=req.name, spec=req.spec, status="running")
    db.add(project)
    db.commit()
    db.refresh(project)

    background_tasks.add_task(run_graph_and_update_db, thread_id, req.spec, project.id)

    return ProjectResponse(id=project.id, thread_id=thread_id, name=project.name,
                           status=project.status, revision_count=0)

async def run_graph_and_update_db(thread_id: str, spec: str, project_db_id: str):
    from app.db.session import SessionLocal
    config = {"configurable": {"thread_id": thread_id}}
    initial_state = {
        "spec": spec, "revision_count": 0, "max_revisions": 3,
        "status": "planning", "review_comments": [], "messages": [],
    }
    try:
        final_state = await graph.ainvoke(initial_state, config=config)
        status = final_state["status"]
    except Exception as e:
        status = "failed"
        final_state = {"revision_count": 0}
        # log the exception with your preferred logging setup

    db = SessionLocal()
    project = db.query(Project).get(project_db_id)
    project.status = status
    project.revision_count = final_state.get("revision_count", 0)
    db.commit()
    db.close()

@router.get("/{project_id}", response_model=ProjectResponse)
async def get_project(project_id: str, db: Session = Depends(get_db)):
    project = db.query(Project).get(project_id)
    if not project:
        raise HTTPException(404, "Project not found")
    return ProjectResponse(id=project.id, thread_id=project.thread_id, name=project.name,
                           status=project.status, revision_count=project.revision_count,
                           repo_url=project.repo_url)

@router.get("/{project_id}/state")
async def get_project_full_state(project_id: str, db: Session = Depends(get_db)):
    """Return the full LangGraph state (files, test results, review comments,
    message trace) for inspection/debugging."""
    project = db.query(Project).get(project_id)
    if not project:

```

```
        raise HTTPException(404, "Project not found")
    config = {"configurable": {"thread_id": project.thread_id}}
    snapshot = graph.get_state(config)
    return snapshot.values
```

10.3 App endpoint

app/main.py :

```
from dotenv import load_dotenv
load_dotenv()

from fastapi import FastAPI
from app.api.routes_projects import router as projects_router

app = FastAPI(title="AI Software Engineering Team")
app.include_router(projects_router)

@app.get("/health")
async def health():
    return {"status": "ok"}
```

Run it:

```
uvicorn app.main:app --reload --port 8000
```

Important: running the graph as a `BackgroundTasks` job is fine for a prototype, but it dies if the FastAPI process restarts mid-run. For anything beyond local testing, move `run_graph_and_update_db` into a real task queue (Celery, RQ, or Dramatiq) backed by Redis or Postgres, so runs survive process restarts and you can scale workers independently of the API.

11. Part H — Running It End-to-End

11.1 Manual smoke test (no API layer)

`scripts/run_sample_project.py`:

```
from dotenv import load_dotenv
load_dotenv()

from app.graph.build_graph import build_graph
import uuid, json

graph = build_graph()

spec = """
Build a small task-tracking API: users can create projects, add tasks to a
project with a title/description/due date, mark tasks complete, and list
tasks filtered by status. Include a minimal web UI to view and manage tasks.
"""

config = {"configurable": {"thread_id": str(uuid.uuid4())}}
initial_state = {
    "spec": spec, "revision_count": 0, "max_revisions": 3,
    "status": "planning", "review_comments": [], "messages": [],
}

for step in graph.stream(initial_state, config=config, stream_mode="values"):
    print("----", step.get("status"), "----")
    for m in step.get("messages", [])[-1:]:
        print(m)

final = graph.get_state(config).values
print(json.dumps({"status": final["status"], "revisions": final["revision_count"]}, indent=2))
```

Run:

```
python scripts/run_sample_project.py
```

`graph.stream(..., stream_mode="values")` yields the full state after every node — this is the fastest way to watch the run progress node-by-node in your terminal before wiring up the API/streaming layer.

11.2 Testing the revision loop specifically

Force a QA failure to verify the loop and `max_revisions` cutoff both work before trusting it against real specs: temporarily hardcode `qa_engineer_node` to return `passed=False`, run the script, and confirm the graph makes exactly `max_revisions` attempts through `backend_dev` before ending with `status="failed"`.

11.3 Testing the sandbox in isolation

Before running the full graph, validate `run_in_sandbox` directly:

```
from app.tools.sandbox_runner import run_in_sandbox

result = run_in_sandbox(
    code_files={
        "main.py": "def add(a, b): return a + b",
        "test_main.py": "from main import add\ndef test_add():\n    assert add(2,3) == 5",
    },
    test_command="pytest -q",
)
print(result)
```

You should see `{"passed": True, "total": 1, "failed": 0, ...}`. Then try a failing test and a deliberately malicious one (e.g. `import os; os.system("curl evil.com")`) to confirm `network_disabled=True` actually blocks it.

12. Part I — Dockerized Deployment

`docker-compose.yml` — brings up Postgres, the FastAPI app, and makes the sandbox image available to the app container via the Docker socket:

```
version: "3.9"

services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_DB: ai_swe_team
      POSTGRES_PASSWORD: postgres
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data

  app:
    build:
      context: .
      dockerfile: Dockerfile
    env_file: .env
    depends_on:
      - postgres
    ports:
      - "8000:8000"
    volumes:
      # Lets the app container launch sibling sandbox containers on the host
      - /var/run/docker.sock:/var/run/docker.sock

volumes:
  pgdata:
```

`Dockerfile` (for the app itself, distinct from `Dockerfile.sandbox`):

```
FROM python:3.12-slim

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Build and run:

```
docker build -t ai-swe-sandbox:latest -f Dockerfile.sandbox .
docker compose up --build
```

Note on mounting the Docker socket: giving the app container access to `/var/run/docker.sock` lets it launch sibling sandbox containers on the host daemon — convenient, but it means anything that can execute code inside the app container effectively has host-level Docker access. For a genuinely untrusted multi-tenant deployment, replace this with a dedicated, network-isolated "sandbox executor" service (e.g. a separate VM, or a managed code-execution API) rather than sharing the host's Docker socket.

13. Part J — Observability & Debugging

- **LangSmith** (optional, from the LangChain team) — set `LANGCHAIN_TRACING_V2=true` and `LANGCHAIN_API_KEY=...` in `.env` and every node's LLM calls, tool calls, and timings are traced automatically. This is the fastest way to see *why* an agent produced a bad output, since you get the exact prompt, tool calls, and response for every step.
 - `graph.get_state(config)` — inspect the full `ProjectState` for any thread at any point, as shown in the `/projects/{id}/state` route.
 - `graph.get_state_history(config)` — walk every checkpoint for a thread, useful for building a "replay this run" debugging view.
 - **Structured logging per node** — log `{node_name, thread_id, duration_ms, tokens_used}` from a lightweight wrapper around each node function, so you can spot which agent is slow or expensive across many runs.
 - **Sandbox logs** — always persist the raw `logs` field from `run_in_sandbox` (capped, as shown, to avoid bloating state) so you can see exactly what QA saw when deciding pass/fail.
-

14. Common Pitfalls & Best Practices

Pitfall	Why it happens	Fix
Unbounded revision loops	Forgetting to gate a conditional edge on <code>revision_count</code>	Always route through a counter check before looping back; treat exhaustion as "failed", not silent retry
Agent-generated code runs on the host	Wiring "Python execution" as a generic shell tool	Route <i>all</i> execution through the Docker sandbox in Part F — no exceptions, even for "just checking syntax"
Free-text outputs break routing	Parsing prose with regex to decide pass/fail	Force <code>with_structured_output</code> / tool-calling JSON schemas for every field the graph logic depends on
Parallel nodes clobber shared state	Two nodes both write to the same key with no reducer	Give shared keys a reducer (<code>Annotated[list, operator.add]</code>) or make sure parallel nodes write disjoint keys
Context window blowup	Passing entire file trees + full message history into every prompt	Pass only what a given agent needs (e.g. backend dev doesn't need frontend files); summarize <code>messages</code> periodically instead of passing raw history
Hallucinated library APIs	LLM writes plausible-looking but wrong API calls	Wire the docs-retrieval RAG tool (6.4) into every code-writing agent, and have QA's sandbox execution be the actual source of truth, not the LLM's self-assessment
Losing runs on process restart	Using <code>BackgroundTasks</code> for long-running graph execution	Move to Celery/RQ/Dramatiq once beyond prototyping; keep the Postgres checkpointer either way so state itself survives
Docker socket exposure	Mounting <code>/var/run/docker.sock</code> into the app container for convenience	Fine for local/trusted use; use a network-isolated execution service for untrusted multi-tenant deployments
Runaway cost	Every node call uses your strongest (most expensive) model	Reserve Sonnet/Opus-tier models for PM/Architect/Reviewer judgment calls; use a cheaper/faster model for QA test generation and repetitive dev-loop iterations
No test for the "it should fail" path	Only testing the happy path end-to-end	Explicitly test that QA failures and reviewer rejections correctly loop and correctly terminate at <code>max_revisions</code> (11.2)

15. Appendix

15.1 Full file tree (recap)

```
ai-swe-team/
├── .env
├── requirements.txt
├── docker-compose.yml
├── Dockerfile
├── Dockerfile.sandbox
├── app/
│   ├── main.py
│   ├── config.py
│   ├── api/
│   │   ├── routes_projects.py
│   │   └── schemas.py
│   ├── graph/
│   │   ├── state.py
│   │   ├── build_graph.py
│   │   └── checkpointer.py
│   ├── agents/
│   │   ├── product_manager.py
│   │   ├── architect.py
│   │   ├── backend_dev.py
│   │   ├── frontend_dev.py
│   │   ├── qa_engineer.py
│   │   ├── code_reviewer.py
│   │   └── _qa_test_writer.py
│   ├── tools/
│   │   ├── mcp_config.py
│   │   ├── sandbox.py
│   │   ├── sandbox_runner.py
│   │   ├── git_tool.py
│   │   ├── docs_retrieval.py
│   │   ├── docs_mcp_server.py
│   │   └── browser_tool.py
│   └── db/
│       ├── models.py
│       ├── session.py
│       └── migrations/
├── tests/
│   └── test_graph.py
└── scripts/
    ├── init_db.py
    └── run_sample_project.py
```

15.2 requirements.txt (starting point)

```
langgraph
langchain
langchain-anthropic
langchain-openai
langchain-mcp-adapters
langgraph-checkpoint-postgres
fastapi
uvicorn[standard]
psycopyg[binary]
psycopyg-pool
docker
gitpython
pydantic
python-dotenv
chromadb
playwright
sqlalchemy
alembic
mcp-server-git
```

15.3 .env.example

```
ANTHROPIC_API_KEY=
OPENAI_API_KEY=
GOOGLE_API_KEY=

GITHUB_TOKEN=
GITHUB_DEFAULT_ORG=

POSTGRES_HOST=localhost
POSTGRES_PORT=5432
POSTGRES_DB=ai_swe_team
POSTGRES_USER=postgres
POSTGRES_PASSWORD=postgres
DATABASE_URL=postgresql://postgres:postgres@localhost:5432/ai_swe_team

SANDBOX_IMAGE=ai-swe-sandbox:latest
SANDBOX_NETWORK_DISABLED=true

LANGCHAIN_TRACING_V2=false
LANGCHAIN_API_KEY=
```

15.4 Suggested build order (recap from the original design discussion)

1. PM → Architect only; verify structured outputs flow correctly.
2. Add Backend Dev + the Docker sandbox — get isolation right before anything else.
3. Add QA + the revision loop — validate looping and the `max_revisions` cutoff.
4. Add Git/GitHub tools so QA-passed code produces a real PR.
5. Add Frontend Dev + the Browser tool, parallelized with Backend Dev.
6. Add the Code Reviewer as the final gate before merge.
7. Wrap everything in FastAPI + Postgres for persistence and external triggering.
8. Move background execution to a real task queue; add LangSmith tracing; harden the sandbox for production.

End of guide.